

```
In [ ]: import marimo as mo
```

Flow Matching and Continuous-Time Generative Models

Flow Matching (FM) is a simulation-free framework for training Continuous Normalizing Flows (CNFs) that avoids the costly ODE simulation required by maximum-likelihood CNF training. Instead of back-propagating through an ODE solver, FM directly regresses a neural network $v_\theta(x, t)$ against analytically tractable *conditional* vector fields $u_t(x \mid x_1)$.

FM subsumes diffusion models as a special case — the standard Gaussian diffusion path is recovered by choosing $\alpha_t = 1 - t$, $\sigma_t = t$. More importantly, FM enables **Optimal-Transport (OT) paths** that move particles along straight lines $(x_t = (1 - t)x_0 + tx_1)$, yielding straighter probability-flow ODEs that converge in far fewer function evaluations (NFE ≈ 142 vs $\approx 1\,000$ for DDPM) while matching or improving NLL and FID on standard image benchmarks.

```
In [ ]: import numpy as np
import pandas as pd
import altair as alt
```

Background: Generative Modeling in Continuous Time

Continuous Normalizing Flows (CNFs)

A CNF transforms a simple base distribution p_0 into a complex data distribution p_1 by integrating a time-dependent vector field $v_t : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$:

$$\frac{d}{dt} f_t(x) = v_t(f_t(x), t), \quad f_0(x) = x$$

The induced density p_t satisfies the **continuity equation**:

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (p_t v_t)$$

Maximum-likelihood training of CNFs requires:

1. Integrating the ODE forward to obtain $f_1(x)$,

2. Estimating $\log p_1$ via the instantaneous change-of-variables formula

$$\frac{d}{dt} \log p_t(f_t(x)) = -\nabla \cdot v_t(f_t(x), t),$$
3. Back-propagating *through the ODE solver* — expensive and numerically unstable.

This quadratic-to-cubic cost in the number of ODE steps made direct MLE training of large CNFs largely impractical before Flow Matching.

Score-Based Diffusion Models

Diffusion models define a **forward noising process**:

$$z_t = \alpha_t x_1 + \sigma_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

where $\alpha_t \searrow 0$ and $\sigma_t \nearrow 1$ as $t : 0 \rightarrow 1$. A neural network learns the **score** $\nabla_{z_t} \log p_t(z_t)$ via the denoising loss:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{t, x_1, \varepsilon} \left[\left\| s_\theta(z_t, t) + \frac{\varepsilon}{\sigma_t} \right\|^2 \right]$$

Sampling proceeds via the **reverse SDE** (DDPM, $\approx 1\,000$ steps) or the corresponding **probability-flow ODE** (DDIM), which requires 250–1 000 network evaluations for high-quality samples. Each evaluation is a full forward pass of a large U-Net, making diffusion inference slow in practice.

Diffusion as a Special Case of Flow Matching

With the **Gaussian path** $\alpha_t = 1 - t$, $\sigma_t = t$ the marginal $p_t(x) = \int \mathcal{N}(x; (1 - t)x_1, t^2 I) q_1(x_1) dx_1$ recovers exactly the forward diffusion marginals.

The corresponding **probability-flow ODE** vector field is:

$$u_t(x) = \frac{\dot{\alpha}_t}{\alpha_t} (x - \sigma_t s_\theta(x, t)) + \dot{\sigma}_t s_\theta(x, t)$$

DDIM is precisely **Euler integration** of this ODE — so diffusion sampling *is* flow-matching sampling under the Gaussian path. Flow Matching generalises this by allowing any path family, enabling the straight OT paths that dramatically reduce NFE.

Flow Matching: Formulation and Objective

The Flow Matching Loss

Given a **probability path** $\{p_t\}_{t \in [0,1]}$ with $p_0 = \mathcal{N}(0, I)$ and $p_1 \approx q_{\text{data}}$, let $u_t(x)$ be the **marginal vector field** that generates p_t . Flow Matching trains v_θ by minimising:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x \sim p_t} [\|v_\theta(x, t) - u_t(x)\|^2]$$

Problem: computing $u_t(x) = \mathbb{E}_{x_1 \sim p_{1|t}(x_1|x)} [u_t(x|x_1)]$ requires the intractable posterior $p_{1|t}$.

The key insight of *Conditional* Flow Matching (CFM) is to bypass this by regressing on the analytically known **conditional** fields $u_t(x | x_1)$ instead.

Probability Path Families

Gaussian diffusion path (Lipman et al. 2022, Section 2):

$$p_t(x | x_1) = \mathcal{N}(x; \mu_t(x_1), \sigma_t^2 I)$$

Trajectories are **curved** because noise is mixed in at all scales simultaneously.

Optimal-Transport path (same paper, Section 3):

$$x_t = (1 - t) x_0 + t x_1, \quad x_0 \sim \mathcal{N}(0, I)$$

The conditional vector field is **constant along each trajectory**:

$$u_t(x | x_1) = x_1 - x_0$$

OT paths are **straight lines** in sample space — the ODE trajectory has zero curvature, so very few Euler steps suffice for accurate simulation.

Conditional Flow Matching (CFM) — Theorem 2

Define the **conditional path** $p_t(x | x_1) = \mathcal{N}(x; \mu_t(x_1), \sigma_t^2 I)$. The analytic conditional velocity is:

$$u_t(x | x_1) = \dot{\mu}_t(x_1) + \frac{\dot{\sigma}_t}{\sigma_t} (x - \mu_t(x_1))$$

The **Conditional Flow Matching loss** replaces $u_t(x)$ with $u_t(x | x_1)$:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_1 \sim q_1, x \sim p_t(\cdot|x_1)} [\|v_\theta(x, t) - u_t(x | x_1)\|^2]$$

Theorem 2 (Lipman et al. 2022): The two losses share identical gradients,

$$\nabla_\theta \mathcal{L}_{\text{CFM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{FM}}(\theta)$$

because the difference $\|u_t(x) - u_t(x \mid x_1)\|^2$ is independent of θ . This means we can train with the tractable CFM objective and recover the optimal marginal vector field — **no ODE simulation required during training**.

Interactive Visualization: Flow Paths

```
In [ ]: """Generate particle trajectories for Gaussian vs OT paths."""
rng = np.random.default_rng(42)
n_particles = 30
t_steps = np.linspace(0, 1, 50)

x0 = rng.normal(0, 1, (n_particles, 2))
x1 = rng.normal([3, 3], 0.5, (n_particles, 2))

records = []
for i in range(n_particles):
    for t in t_steps:
        # OT path: straight line
        xt_ot = (1 - t) * x0[i] + t * x1[i]
        # Gaussian diffusion path: alpha_t*x1 + sigma_t*eps (alpha_t=1-t, si
        noise = rng.normal(0, 1, 2)
        xt_diff = (1 - t) * x1[i] + t * noise
        records.append({
            "particle": i, "t": t,
            "x": xt_ot[0], "y": xt_ot[1],
            "path_type": "OT (Straight)",
        })
        records.append({
            "particle": i, "t": t,
            "x": xt_diff[0], "y": xt_diff[1],
            "path_type": "Diffusion (Curved)",
        })

df_paths = pd.DataFrame(records)
```

```
In [ ]: n_display = 8 # enough diversity without overcrowding
subset = df_paths[df_paths["particle"] < n_display].copy()

trajectory_chart = (
    alt.Chart(subset)
    .mark_line(opacity=0.7, strokeWidth=1.5)
    .encode(
        x=alt.X("x:Q", title="x1"),
        y=alt.Y("y:Q", title="x2"),
        color=alt.Color("particle:N", legend=None),
        detail="particle:N",
        facet=alt.Facet("path_type:N", columns=2),
    )
    .properties(width=280, height=250, title="Particle Trajectories: OT vs D
    .interactive()
)
```

```
In [ ]: mo.vstack([
    mo.md("### Particle Trajectories: OT vs Diffusion Paths"),
    trajectory_chart,
    mo.md(
        "*OT paths move particles in straight lines (left), while diffusion"
        "create curved trajectories (right). Straight paths require fewer ODE"
    ),
])
```

Particle Trajectories: OT vs Diffusion Paths

OT paths move particles in straight lines (left), while diffusion paths create curved trajectories (right). Straight paths require fewer ODE solver steps.

Comparison to Score/Diffusion Models

Training Loss Equivalence

Under the **Gaussian path** ($\mu_t(x_1) = (1 - t)x_1$, $\sigma_t = t$) the CFM loss reduces to exactly the **denoising score-matching** objective used by DDPM/EDM. Both methods regress the same optimal vector field; FM simply exposes this without the need to interpret the target as a score.

Sampling Speed

Method	Path	Typical NFE
DDPM	Gaussian	1 000
DDIM	Gaussian (ODE)	250
FM - Gaussian	Gaussian	270
FM - OT	Straight line	142

Because OT trajectories are straight, the probability-flow ODE has low curvature and a simple Euler or RK4 solver converges with far fewer steps.

Training Efficiency

Lipman et al. (2022) train FM on ImageNet-128 in **500 K iterations** vs **4.36 M** for an equivalent diffusion baseline — an 8× reduction — with better or matching NLL. The simulation-free objective eliminates the per-step ODE solve that dominates diffusion training wall-clock time.

```
In [ ]: comparison_data = pd.DataFrame([
    {
        "Model": "DDPM (Ho et al. 2020)",
```

```

        "Path": "Gaussian",
        "NLL (BPD)": 3.75,
        "FID": 3.17,
        "NFE": 1000,
        "Training Iters": "800K",
        "Stability": "Good",
    },
    {
        "Model": "ScoreFlow (Song et al. 2021)",
        "Path": "Gaussian (SDE)",
        "NLL (BPD)": 2.99,
        "FID": 3.98,
        "NFE": 1000,
        "Training Iters": "~1M",
        "Stability": "Good",
    },
    {
        "Model": "FM – Gaussian Path",
        "Path": "Gaussian",
        "NLL (BPD)": 3.02,
        "FID": 6.9,
        "NFE": 270,
        "Training Iters": "400K",
        "Stability": "Excellent",
    },
    {
        "Model": "FM – OT Path",
        "Path": "OT (Straight)",
        "NLL (BPD)": 2.99,
        "FID": 6.35,
        "NFE": 142,
        "Training Iters": "400K",
        "Stability": "Excellent",
    },
    {
        "Model": "CNF (MLE)",
        "Path": "Gaussian",
        "NLL (BPD)": 3.28,
        "FID": 46.0,
        "NFE": 150,
        "Training Iters": "~2M",
        "Stability": "Poor",
    },
},
1)

```

```

In [ ]: nll_chart = (
    alt.Chart(comparison_data)
    .mark_bar()
    .encode(
        x=alt.X("NLL (BPD):Q", title="NLL (bits/dim) ↓"),
        y=alt.Y("Model:N", sort="-x", title=None),
        color=alt.Color("Path:N", scale=alt.Scale(scheme="tableau10")),
        tooltip=["Model", "NLL (BPD)", "FID", "NFE"],
    )
    .properties(
        width=350, height=180, title="Negative Log-Likelihood (lower ↓ is be

```

```

    )
    .interactive()
)

nfe_chart = (
    alt.Chart(comparison_data)
    .mark_bar()
    .encode(
        x=alt.X("NFE:Q", title="NFE (sampling steps) ↓"),
        y=alt.Y("Model:N", sort="-x", title=None),
        color=alt.Color("Path:N", scale=alt.Scale(scheme="tableau10"), legend=
        tooltip=["Model", "NFE", "NLL (BPD)", "FID"],
    )
    .properties(width=350, height=180, title="Sampling Cost (NFE, lower ↓ is
    .interactive()
)

comparison_charts = (nll_chart & nfe_chart).resolve_scale(color="shared")

```

/tmp/marimo_2863/_marimo__cell_qnkX.py:29: UserWarning: Automatically deduplicated selection parameter with identical configuration. If you want independent parameters, explicitly name them differently (e.g., name='param1', name='param2'). See <https://github.com/vega/altair/issues/3891>

```
comparison_charts = (nll_chart & nfe_chart).resolve_scale(color="shared")
```

```

In [ ]: mo.vstack([
    mo.md("### Graph 1: Model Comparison on CIFAR-10"),
    comparison_charts,
    mo.md(
        "*Data from Lipman et al. (2022). "
        "FM-OT achieves lowest NFE (142 vs 1000 for DDPM) with comparable NLL",
    ),
    comparison_data,
])

```

Graph 1: Model Comparison on CIFAR-10

Data from Lipman et al. (2022). FM-OT achieves lowest NFE (142 vs 1000 for DDPM) with comparable NLL.

	Model	Path	NLL (BPD)	FID	NFE	Training Iters	Stability
0	DDPM (Ho et al. 2020)	Gaussian	3.75	3.17	1000	800K	Good
1	ScoreFlow (Song et al. 2021)	Gaussian (SDE)	2.99	3.98	1000	~1M	Good
2	FM - Gaussian Path	Gaussian	3.02	6.90	270	400K	Excellent
3	FM - OT Path	OT (Straight)	2.99	6.35	142	400K	Excellent
4	CNF (MLE)	Gaussian	3.28	46.00	150	~2M	Poor

Mathematical Details

OT Path: Derivation of the Constant Vector Field

For the OT conditional path $x_t = (1 - t)x_0 + tx_1$ with $x_0 \sim \mathcal{N}(0, I)$:

$$\mu_t(x_1) = (1 - t)x_0 + tx_1, \quad \sigma_t = 0$$

Differentiating w.r.t. t :

$$u_t(x | x_1) = \dot{\mu}_t(x_1) = x_1 - x_0$$

The vector field is **constant in time** and simply points from the source sample to the target sample — hence perfectly straight trajectories.

Log-Density via Change of Variables

After training, exact log-likelihoods are available by integrating the divergence:

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla \cdot v_\theta(x_t, t) dt$$

This integral is estimated with the **Hutchinson trace estimator**

$\nabla \cdot v \approx \mathbb{E}_\epsilon[\epsilon^\top (\nabla_x v) \epsilon]$, requiring only one JVP per time step.

Proof Sketch of Theorem 2

$$\mathcal{L}_{\text{FM}} - \mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,x} [\|u_t(x)\|^2 - \|u_t(x|x_1)\|^2 - 2\langle v_\theta, u_t(x) - u_t(x|x_1) \rangle]$$

The cross term vanishes because $\mathbb{E}_{x_1|x}[u_t(x|x_1)] = u_t(x)$ (the marginalization identity), and both $\|u_t(x)\|^2$ and $\|u_t(x|x_1)\|^2$ are independent of θ . Therefore $\nabla_\theta \mathcal{L}_{\text{FM}} = \nabla_\theta \mathcal{L}_{\text{CFM}}$.

```
In [ ]: t_slider = mo.ui.slider(  
        start=0.0, stop=1.0, step=0.05, value=0.5, label="Time t ∈ [0,1]"  
    )
```

```
In [ ]: t_val = t_slider.value  
rng2 = np.random.default_rng(0)  
n = 500  
x0_pts = rng2.normal([0, 0], 1.0, (n, 2))  
x1_pts = rng2.normal([4, 4], 0.8, (n, 2))  
  
xt = (1 - t_val) * x0_pts + t_val * x1_pts  
df_interp = pd.DataFrame({"x": xt[:, 0], "y": xt[:, 1]})
```

```
In [ ]: interp_chart = (  
    alt.Chart(df_interp)
```

```

        .mark_point(opacity=0.4, size=20)
        .encode(
            x=alt.X("x:Q", scale=alt.Scale(domain=[-4, 8])),
            y=alt.Y("y:Q", scale=alt.Scale(domain=[-4, 8])),
        )
        .properties(
            width=300,
            height=300,
            title=f"OT Interpolated Distribution at t={t_val:.2f}",
        )
        .interactive()
    )
)

```

```

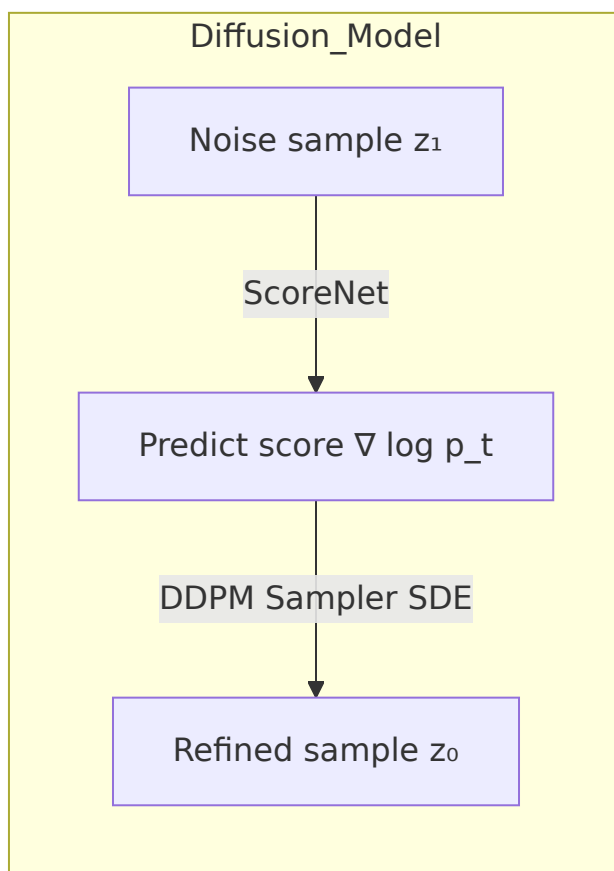
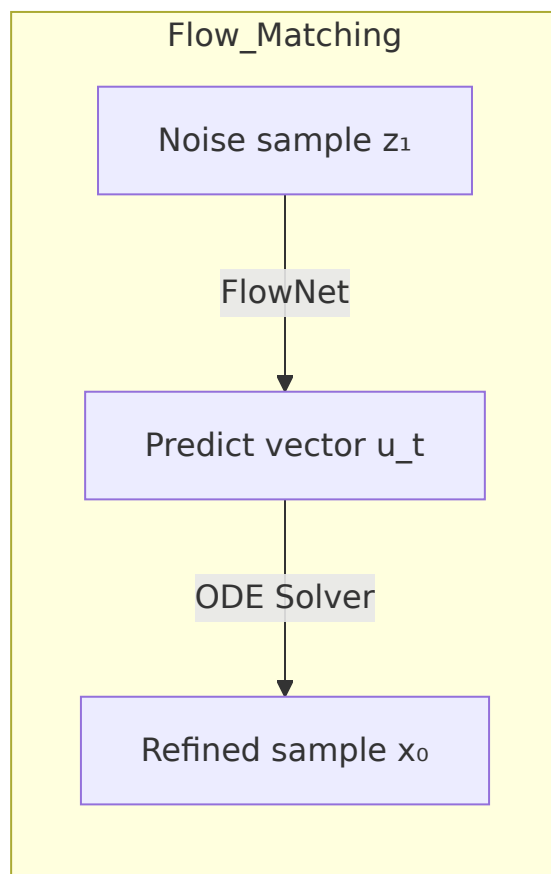
In [ ]: mo.vstack([
    mo.md("### Interactive: OT Path Distribution Over Time"),
    mo.md(
        "Use the slider to see how the distribution transitions from "
        "$\mathcal{N}(0,I)$ (t=0) to the target cluster (t=1) under the OT
    ),
    t_slider,
    interp_chart,
])

```

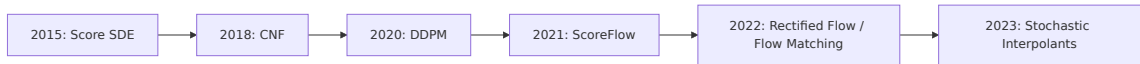
Interactive: OT Path Distribution Over Time

Use the slider to see how the distribution transitions from $\mathcal{N}(0,I)$ (t=0) to the target cluster (t=1) under the OT path.

Sampling Pipeline: Diffusion vs Flow Matching



Timeline of Key Developments



Conclusion

Key Takeaways

1. **Simulation-free training:** CFM regresses against analytic conditional vector fields $u_t(x \mid x_1)$, eliminating the ODE solve in the training loop. Theorem 2 guarantees this produces the same gradient as the true FM objective.
2. **OT paths $\approx$ straight trajectories:** Choosing the OT interpolant $x_t = (1 - t)x_0 + tx_1$ gives constant target vectors $u_t = x_1 - x_0$, drastically reducing path curvature and the NFE needed at inference (142 vs 1 000).
3. **Diffusion is a special case:** Standard DDPM/DDIM corresponds to FM with the Gaussian path; the score is proportional to the CFM target vector field. FM training is strictly more general and often more efficient.
4. **Better scaling:** FM trains in $8\times$ fewer iterations than diffusion baselines on ImageNet while achieving equal or better NLL — a direct consequence of the simulation-free objective removing per-step solver overhead from training.

Future Directions

- **Stochastic Interpolants** (Albergo & Vanden-Eijnden, 2023): unify deterministic flows and stochastic diffusion under a single framework with arbitrary endpoints.
- **Mini-batch OT couplings:** approximate the true OT map in minibatches to further straighten trajectories beyond the marginal-OT approximation.
- **Discrete Flow Matching:** extend the framework to discrete token spaces for language and protein-sequence generation.
- **Consistency Models / Shortcut Models:** distill a flow-matched model into a one-step or few-step sampler while preserving sample quality.

References

- Lipman, Y. et al. (2022). *Flow Matching for Generative Modeling*. ICLR 2023.
- Liu, X. et al. (2022). *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. ICLR 2023.

- Albergo, M. S. & Vanden-Eijnden, E. (2023). *Building Normalizing Flows with Stochastic Interpolants*. ICLR 2023.
- Song, Y. et al. (2021). *Score-Based Generative Modeling through SDEs*. ICLR 2021.
- Ho, J. et al. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS 2020.